

Student

Lab 1 delivered a runnable Mango Mart star schema. Now let's **use it**.

ChatGPT • AI Assistant

Lab 2 is the payoff - you'll write the exact SQL that answers each of the top-10 analytical questions Mango Mart's leadership listed back in Lab 1. Every query joins your `fact_sales` to 2-4 dimensions and turns raw rows into a business KPI. Structure of the lab:

1. **Lab setup** - recap of Mango Mart schema; sample data + query conventions.
2. **KPI 1 & 2** - Revenue by category × region × month AND top products.
3. **KPI 3** - Customer lifetime value by segment.
4. **KPI 4** - Promo effectiveness (uplift analysis).
5. **KPI 5** - Year-over-year growth by store format.
6. **KPI 6** - Cohort retention analysis.
7. **KPI 7 & 8** - Store performance + employee attribution.
8. **KPI 9** - Market basket analysis (co-purchases).
9. **KPI 10 + performance tips** - Return rates + production-ready query patterns.
10. **Lab review & summary**.

CONCEPT CARD

By the end of this lab you'll have:

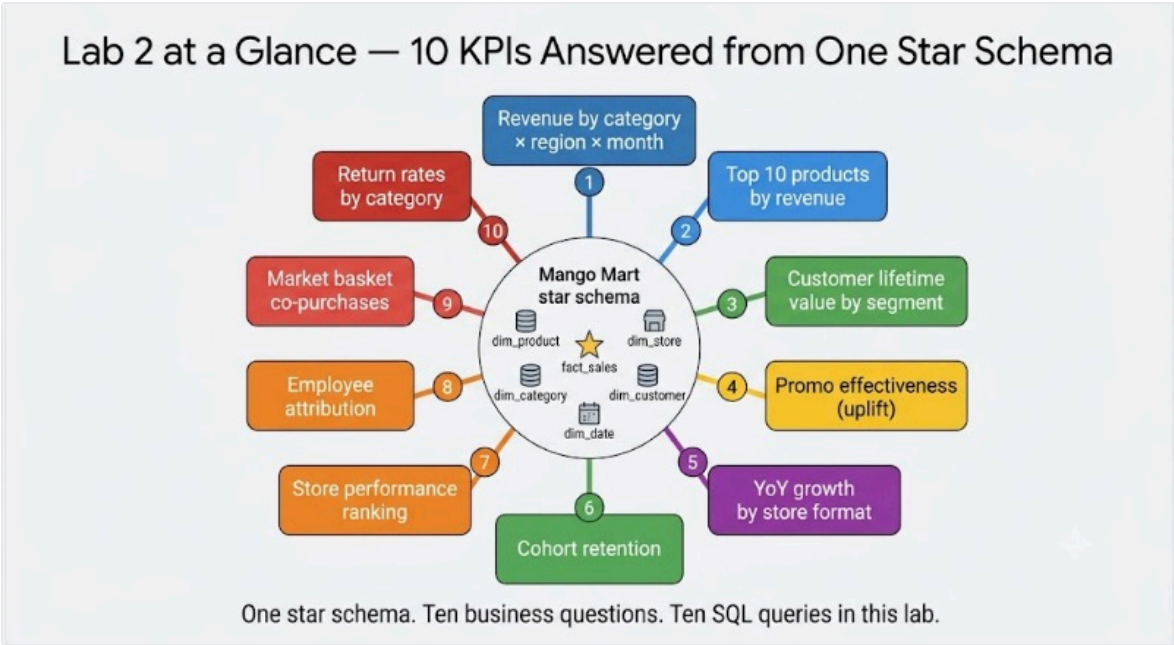
- **10 working KPI queries** you can copy directly into any warehouse.
- **Fluency with window functions** (RANK, LAG, PERCENTILE_CONT).
- **Comfort with CTEs** for multi-step analytical logic.
- **A checklist of performance patterns** for production analytics SQL.

SUMMARY CARD

The queries in this lab exercise every core technique of analytical SQL: joins, filters, group-bys, window functions, CTEs, self-joins, and time-series arithmetic. If you can write these ten, you can write any KPI a business asks for.

[AI Prompt] Warm-up (paste into any AI chatbot): *"Given a Kimball star schema with fact_sales at line-item grain and dimensions dim_date, dim_customer, dim_product, dim_store, dim_promotion, dim_employee, write the SQL for 'revenue by category*

and region per month'. Return the answer with a step-by-step explanation of each JOIN and GROUP BY."



Lab 2 at a Glance — 10 KPIs Answered from One Star Schema

Top-10 KPI Query Techniques at a Glance		
KPI	Query Techniques Used	Difficulty
1 — Revenue by category × region × month	3-way join + GROUP BY	Beginner
2 — Top products by revenue and units	GROUP BY + RANK() window	Beginner
3 — Customer LTV by segment	CTE + AVG + PERCENTILE_CONT	Intermediate
4 — Promo effectiveness (uplift)	Filtered aggregates + conditional AVG	Intermediate
5 — Year-over-year growth	LAG() window	Intermediate
6 — Cohort retention	Multi-CTE + DATE_TRUNC + self-derived cohort	Advanced
7 & 8 — Store + employee rankings	GROUP BY + RANK() + PARTITION BY	Beginner
9 — Market basket co-purchases		
10 — Return rates by category		

KPI	Query Techniques Used	Difficulty
9 — Market basket analysis	Self-join on order_id	Intermediate
10 — Return rates by category	Cross-fact join via conformed dims	Intermediate

```
sql – Every query in this lab follows this shape

-- The universal star-schema KPI query pattern:
--
-- SELECT    <dim attributes>, <measure aggregations>
-- FROM      fact_sales s
-- JOIN      dim_...    ON <FK relationship>
-- WHERE     <dim filters> AND <SCD is_current = TRUE where needed>
-- GROUP BY  <dim attributes>
-- ORDER BY  <measure or attribute>
--
-- Every KPI below is a variation on this shape.
-- Master this pattern and you can answer any business question.

-- Setup convention for the rest of this lab:
--   - All queries assume the Lab 1 schema is loaded.
--   - All queries filter dim_customer, dim_product, dim_employee
--     to is_current = TRUE except where historical accuracy matters.
--   - All time filters go through dim_date (never EXTRACT /
DATE_PART).
```

LAB SETUP — SCHEMA RECAP AND QUERY CONVENTIONS



MANGO MART QUERY CONVENTIONS



1. Time filters via `dim_date` (never `EXTRACT / DATE_PART`)



2. `is_current = TRUE` on Type 2 dims (unless historical accuracy needed)



3. `net_amount` for revenue (after discount, before tax)



4. Consistent aliases: **s / p / c / d / st / pr / e**



5. Explicit column lists (no `SELECT *`)

Follow all 5 in every query. Consistency = maintainability.

The Query Convention Cheat Sheet — Applied to Every KPI Below

Student

Before we start writing queries, remind me of the schema.

ChatGPT • AI Assistant

The Mango Mart star from Lab 1 has one central fact and six conformed dimensions. Every query in this lab uses the same table names and column conventions.

CONCEPT CARD

The Mango Mart schema in one line each:

1. ``fact_sales`` - line-item grain; FKs to all 6 dims; measures include *netamount*, *quantity*, *discountamount*, *cost_amount*.
2. ``dim_date`` - YYYYMMDD int PK; *fulldate*, *monthname*, *quarter*, *year*, *isweekend*, *isholiday*.
3. ``dim_customer`` - SCD Type 2; *segment*, *loyaltytier*, *region*, *signupdate*.
4. ``dim_product`` - SCD Type 2; *category*, *brand*, *unit_cost*.
5. ``dim_store`` - *store_format*, *region*.
6. ``dim_promotion`` - Type 0; *discounttype*, *campaign*, *startdate*, *end_date*.
7. ``dim_employee`` - SCD Type 2; *role*, *department*.

What conventions should I follow in every KPI query?

ChatGPT • AI Assistant

EXAMPLE CARD

Five conventions professional analytics SQL always follows:

1. **Time filters via `dim\_date`** - never `EXTRACT(month FROM ...)` ; join and filter on `d.year` , `d.month_name` , etc.
2. **`is\_current = TRUE`** on Type 2 dims when the analyst wants the LATEST attributes (default for BI); omit when the question is genuinely historical.
3. **`net\_amount`, not gross** - Mango Mart reports revenue after promo discount but before tax; use `net_amount` .
4. **Aliases in every JOIN** - `s` for `factsales`, `p` for `dimproduct`, `c` for `dimcustomer`, `d` for `dimdate`, `st` for `dimstore`, `pr` for `dimpromotion`, `e` for `dim_employee`. Consistent aliases make queries readable at glance.
5. **Explicit column lists** - never `SELECT *` . Column stores punish it; readers dislike it.

🔗 ANALOGY

Setting up a lab is like **preparing a mise en place before cooking**. Every ingredient (table alias) is chopped and placed within reach. When the pan is hot (the query starts), you don't fumble.

⚠️ WARNING

Some queries below deliberately turn OFF `is_current = TRUE` filtering because they need historical accuracy (e.g., cohort retention). Read each query's intent before applying blindly.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Filtering by `WHERE order_date > '2026-01-01'` on the fact table directly. Bypasses `dim_date`, hurts partition pruning.

Correct Way: `JOIN dim_date d ... WHERE d.year = 2026` . Enables the warehouse's date-partition pruning and shared calendar attributes.

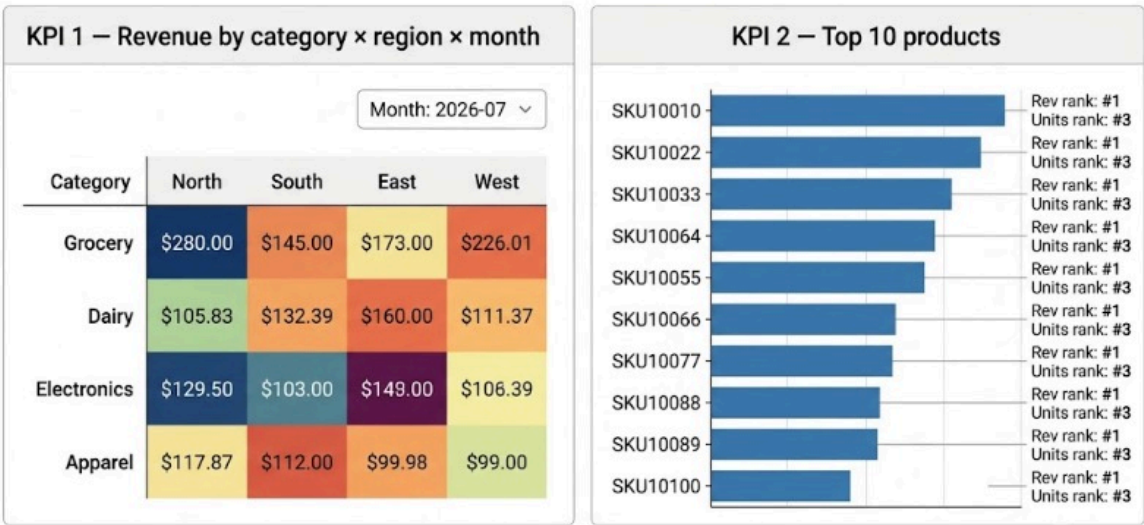
🔥 EXERCISE

Before running any KPI query, verify Lab 1's DDL is loaded and populate 30-50 sample fact rows across 2-3 months, 5-10 customers, and 3-5 products. You'll see the results shape clearly with tiny data.

[AI Prompt] Paste this into any AI chatbot: "For a Kimball star schema on Snowflake, list 10 SQL query conventions that professional analytics teams follow. Include naming, filtering, join style, and performance tips."

KPI 1 & 2 — REVENUE ANALYTICS

ChatGPT • AI Assistant



Two queries. Two lenses on the same data. Both are one join away.
KPI 1 & 2 Result Shape — Revenue Grid + Top-Product Ranking

KPI 1 & 2 Result Shape — Revenue Grid + Top-Product Ranking

```
sql — KPI 1 — Revenue by category × region × month

-- =====
-- KPI 1: Revenue and order count by product category,
--       store region, and calendar month.
-- =====

SELECT
  p.category,
  st.region,
  d.year,
  d.month_name,
  SUM(s.net_amount)           AS revenue,
  COUNT(DISTINCT s.order_id)  AS orders,
  SUM(s.quantity)            AS units_sold
FROM fact_sales      s
JOIN dim_product     p ON p.product_key = s.product_key
```

```
JOIN dim_store      st ON st.store_key = s.store_key
JOIN dim_date       d  ON d.date_key   = s.date_key
WHERE d.year        = 2026
      AND p.is_current = TRUE
GROUP BY p.category, st.region, d.year, d.month_name, d.month_number
ORDER BY d.month_number, p.category, revenue DESC;
```

sql – KPI 2 – Top 10 products by revenue AND top 10 by units

```
-- =====
-- KPI 2: Top 10 products by revenue AND top 10 by units.
--       Products that make both lists are the true winners.
-- =====

WITH product_totals AS (
  SELECT
    p.sku,
    p.product_name,
    p.category,
    SUM(s.net_amount)           AS revenue,
    SUM(s.quantity)            AS units_sold
  FROM fact_sales      s
  JOIN dim_product    p ON p.product_key = s.product_key
  JOIN dim_date       d ON d.date_key   = s.date_key
  WHERE d.year        = 2026
        AND p.is_current = TRUE
  GROUP BY p.sku, p.product_name, p.category
),
ranked AS (
  SELECT *,
         RANK() OVER (ORDER BY revenue      DESC) AS revenue_rank,
         RANK() OVER (ORDER BY units_sold DESC) AS units_rank
  FROM   product_totals
)
SELECT sku, product_name, category,
       revenue, units_sold,
       revenue_rank, units_rank
FROM   ranked
WHERE  revenue_rank ≤ 10
      OR units_rank  ≤ 10
ORDER BY revenue_rank;
```

Let's start with the classic - **revenue by category × region × month**, plus the **top 10 products**.

ChatGPT • AI Assistant

These are the bread-and-butter of retail analytics. Both are three-way joins with grouping, and both introduce a critical pattern you'll reuse in every KPI: **filter through dim_date, group by dimension attributes, aggregate the fact's measures**.

CONCEPT CARD

KPI 1 shape:

1. **Group by:** category (from *dimproduct*), region (from *dimstore*), yearmonth (from *dimdate*).
2. **Aggregate:** SUM(*netamount*) for revenue, COUNT(DISTINCT *orderid*) for orders.
3. **Order by:** year_month ascending, revenue descending.

KPI 2 shape:

1. **Group by:** sku, product_name, category.
2. **Aggregate:** SUM(*net_amount*) for revenue, SUM(*quantity*) for units.
3. **Rank:** two window functions - one by revenue, one by units - to surface top-10 in each.
4. **Filter:** `RANK() ≤ 10` for either dimension.

Student

Why use `RANK()` instead of `LIMIT 10` ?

ChatGPT • AI Assistant

EXAMPLE CARD

`RANK()` gives you:

1. **Two rankings in one query** - top-10 by revenue AND top-10 by units together.
2. **Ties handled correctly** - two products tied at #10 both appear.
3. **Partitioned rankings** - you can also rank within category via PARTITION BY.
4. **No second pass** - LIMIT can't return top-N per group without additional CTEs.

ANALOGY

Ranking with window functions is like **giving out medals**. `LIMIT 10` says "only the first 10 across the finish line." `RANK()` says "here's everyone's medal - bronze, silver, gold - sortable by category too."

⚠ WARNING

Watch out for `DENSE_RANK` vs `RANK` vs `ROW_NUMBER`. Ties in `RANK` skip the next value (1, 1, 3); `DENSE_RANK` doesn't (1, 1, 2); `ROW_NUMBER` never ties. Match to the business's ranking semantics.

⚠ COMMON PITFALL / MISTAKE

Incorrect Way: `WHERE EXTRACT(MONTH FROM s.order_date) = 7` - bypasses `dim_date` and defeats optimization.

Correct Way: `JOIN dim_date d ... WHERE d.year = 2026 AND d.month_number = 7`.

EXAM TIP CARD

Interview classic: "Write a SQL query that returns top-10 products by revenue with ties." Winning approach: "Use `RANK()` in a CTE, filter `WHERE rank <= 10` in the outer query."

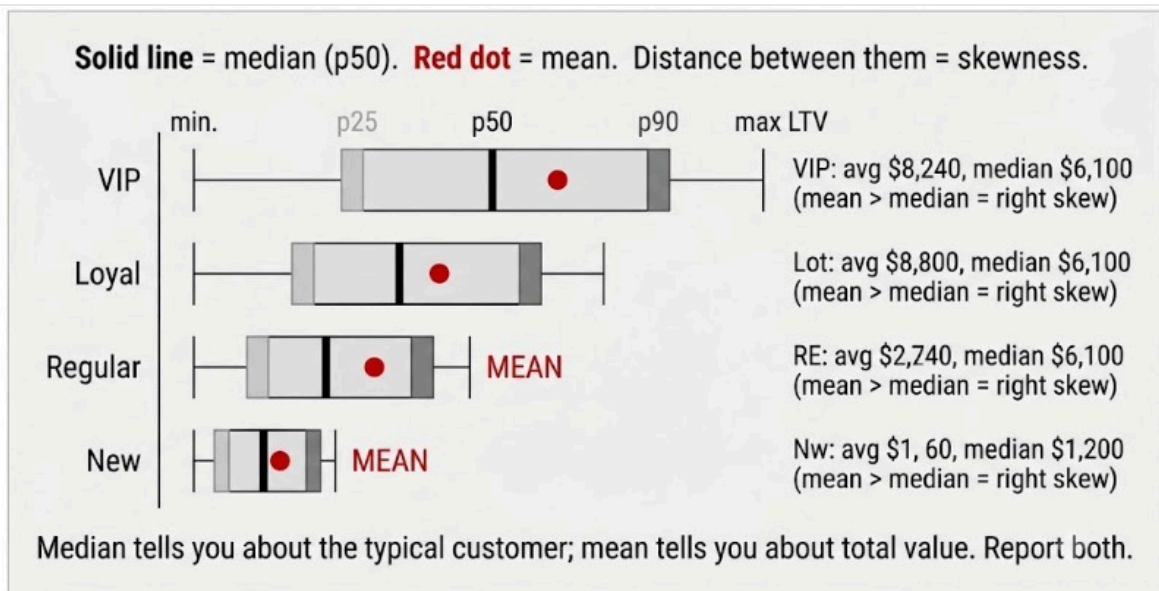
🔥 EXERCISE

Modify KPI 1 to also show `quantity_sold`. Modify KPI 2 to rank within each category (top-3 per category). These are trivial extensions once the base pattern is in place.

[AI Prompt] Paste this into any AI chatbot: "Extend KPI 2 to return the top 3 products in each category by revenue. Show the `DENSE_RANK()` OVER (PARTITION BY category ORDER BY revenue DESC) technique and explain when it's better than a subquery."

KPI 3 — CUSTOMER LIFETIME VALUE BY SEGMENT

ChatGPT • AI Assistant



KPI 3 Result Shape — LTV Distributions Per Segment.

KPI 3 Result Shape — LTV Distributions Per Segment

sql — KPI 3 — Customer LTV distribution per segment

```
-- =====
-- KPI 3: Customer lifetime value (LTV) by segment.
--       Report mean AND median — LTV is heavily skewed.
-- =====

WITH customer_ltv AS (
  -- Layer 1: per-customer lifetime totals
  SELECT
    c.customer_id,
    c.segment,
    SUM(s.net_amount)          AS lifetime_revenue,
    COUNT(DISTINCT s.order_id) AS lifetime_orders,
    MIN(d.full_date)           AS first_order_date,
    MAX(d.full_date)           AS last_order_date
  FROM fact_sales s
  JOIN dim_customer c ON c.customer_key = s.customer_key
  JOIN dim_date d ON d.date_key = s.date_key
  WHERE c.is_current = TRUE
  GROUP BY c.customer_id, c.segment
)
SELECT
  segment,
  COUNT(*)
AS customer_count,
  ROUND(AVG(lifetime_revenue), 2)
AS mean_ltv,
```

```

ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY lifetime_revenue),
2) AS median_ltv,
ROUND(PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY lifetime_revenue),
2) AS p90_ltv,
ROUND(AVG(lifetime_orders), 2)
AS avg_orders_per_customer,
ROUND(AVG(lifetime_revenue) / NULLIF(AVG(lifetime_orders), 0), 2)
AS avg_order_value
FROM customer_ltv
GROUP BY segment
ORDER BY mean_ltv DESC;

```

Student

How do I compute **customer lifetime value (LTV)** and compare across segments?

ChatGPT • AI Assistant

LTV = total revenue generated by a customer since signup. Aggregate all their orders, then group customers by segment and compute AVG + MEDIAN LTV per segment. Median matters here because LTV distributions are heavily right-skewed - a few whales drag the mean way up.

CONCEPT CARD

KPI 3 has two layers:

1. **Layer 1:** per-customer LTV - total revenue, total orders, first order date, last order date.
2. **Layer 2:** per-segment aggregates - AVG(ltv), MEDIAN(ltv) via **PERCENTILE_CONT(0.5)**.

Student

Why MEDIAN specifically?

ChatGPT • AI Assistant

EXAMPLE CARD

Why median beats mean for LTV:

1. LTV distributions are **long-tailed** - 5% of customers often account for 40% of revenue.
2. **Mean gets distorted** by those whales - a segment might look wealthy on average but half its customers are barely active.
3. **Median** answers *"what does the typical customer look like?"* - the honest number.
4. Always report **both** - mean for total value, median for typicality.

🔗 ANALOGY

Reporting only the mean is like **describing house prices by mean when Elon Musk is on the block**. Median tells the real story of what most people paid.

⚠️ WARNING

`PERCENTILE_CONT(0.5)` is standard in Snowflake, BigQuery, Postgres, and Redshift. Older MySQL versions lack it - use ordered aggregates or approximations.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: LTV computed only for customers `WHERE is_current = TRUE` - you lose historical customers who churned but whose LTV still matters for cohort analysis.

Correct Way: Include all customer versions; group by `customer_id` (the natural key that spans all versions).

EXAM TIP CARD

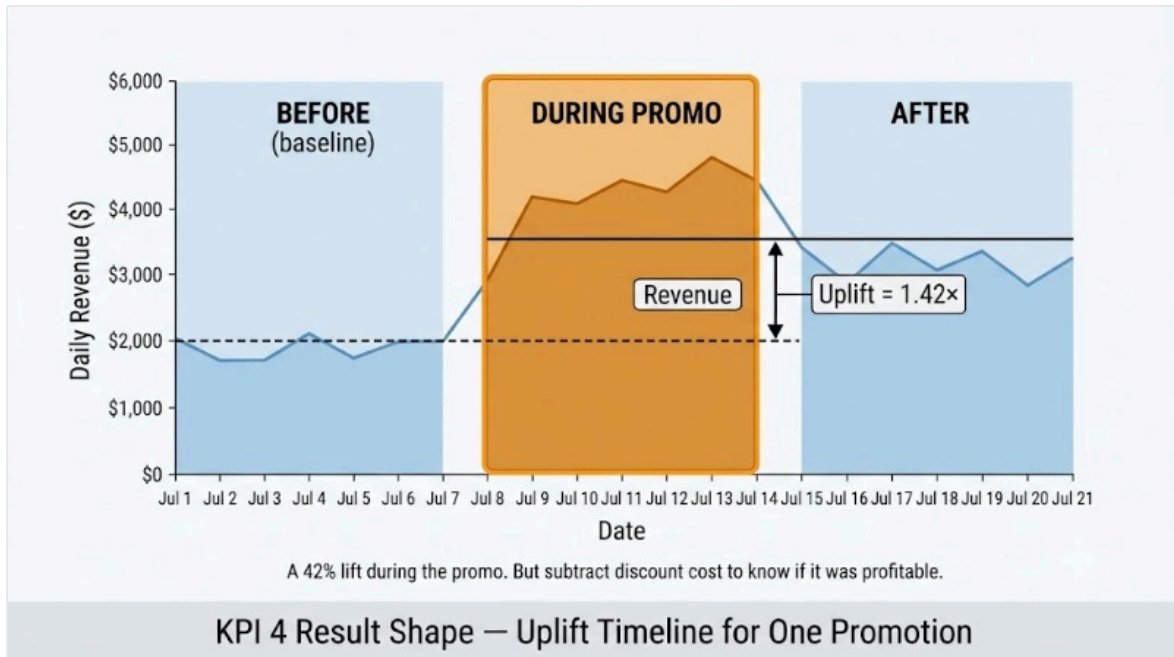
Interview classic: *"What's the difference between using mean and median for customer metrics?"* Winning answer: *"Mean is distorted by outliers; median describes the typical customer. For skewed distributions like LTV, revenue-per-user, or order values, always report both."*

🔥 EXERCISE

Extend KPI 3 to include LTV percentiles: p25, p50 (median), p75, p90, p99. Notice how far apart p50 and p99 are - that's the wealth gap inside each segment.

[AI Prompt] Paste this into any AI chatbot: *"Extend the customer LTV query to include p25, p50, p75, p90, p99 percentiles per segment. Explain what each percentile reveals about the segment's economics."*

KPI 4 — PROMO EFFECTIVENESS (UPLIFT ANALYSIS)



KPI 4 Result Shape — Uplift Timeline for One Promotion

sql — KPI 4 — Promo effectiveness with before/during/after uplift

```
-- =====
-- KPI 4: Promo effectiveness — uplift ratio + discount cost.
-- =====

WITH promo_daily AS (
  -- All sales in a ±7-day window around each promo, per day
  SELECT
    pr.promotion_key,
    pr.promo_name,
    pr.campaign,
    pr.start_date,
    pr.end_date,
    d.full_date,
    SUM(s.net_amount)      AS daily_revenue,
    SUM(s.discount_amount) AS daily_discount
  FROM fact_sales      s
  JOIN dim_promotion   pr ON pr.promotion_key = s.promotion_key
  JOIN dim_date        d  ON d.date_key       = s.date_key
  WHERE d.full_date BETWEEN pr.start_date - INTERVAL '7 days'
                        AND pr.end_date   + INTERVAL '7 days'
  GROUP BY pr.promotion_key, pr.promo_name, pr.campaign,
           pr.start_date, pr.end_date, d.full_date
)
SELECT
  promo_name,
```

```

campaign,
ROUND(AVG(daily_revenue) FILTER (
  WHERE full_date < start_date
), 2)
AS
avg_daily_before,
ROUND(AVG(daily_revenue) FILTER (
  WHERE full_date BETWEEN start_date AND end_date
), 2)
AS
avg_daily_during,
ROUND(AVG(daily_revenue) FILTER (
  WHERE full_date > end_date
), 2)
AS
avg_daily_after,
ROUND(
  AVG(daily_revenue) FILTER (WHERE full_date BETWEEN start_date AND
end_date)
  / NULLIF(AVG(daily_revenue) FILTER (WHERE full_date < start_date),
0),
  2
)
AS
uplift_ratio,
ROUND(SUM(daily_discount) FILTER (
  WHERE full_date BETWEEN start_date AND end_date
), 2)
AS
discount_cost
FROM   promo_daily
GROUP BY promo_name, campaign
ORDER BY uplift_ratio DESC NULLS LAST;

```

Student

How do I measure whether a **promotion actually worked**?

ChatGPT • AI Assistant

The right measure is **uplift**: the ratio of revenue *during* a promo to a comparable *baseline* period. A promo with 1.5× uplift generated 50% more revenue than baseline; a promo with 0.9× uplift arguably cannibalized normal sales.

The query needs three windows per promo: before, during, after.

CONCEPT CARD

KPI 4 structure:

1. **Per-promo daily revenue** - join *factsales through dimpromotion*, filtered to a window around each promo.
2. **Filtered aggregates** - `AVG(daily_revenue) FILTER (WHERE full_date BETWEEN start AND end)` for during; `WHERE full_date < start` for before.
3. **Uplift ratio** - during / before.
4. **Discount cost** - total discount_amount attributed to the promo.
5. **Net uplift** - uplift revenue minus discount cost.

Student

Should baseline be "before" or a full non-promo period?

ChatGPT • AI Assistant

EXAMPLE CARD

Two baseline choices:

1. **Simple baseline** - average daily revenue in the 7 days BEFORE the promo. Easy, but doesn't correct for seasonality.
2. **Seasonal baseline** - average daily revenue in the same 7 days one year earlier. Correct for annual patterns; needs a full year of data.
3. **Cross-sectional baseline** - non-promo products in the same period. Isolates the promo effect from time-based trends.

KPI 4 uses the simple 7-days-before baseline. In production, marketing teams often use all three and triangulate.

🔗 ANALOGY

Uplift analysis is like **A/B testing a shop's window display**. You measure sales with the display (during) vs without (before). The ratio tells you whether the display moved product - or just rearranged which day people bought.

⚠️ WARNING

Uplift can be **misleading** if the promo drove customers to buy the SAME thing they would have bought anyway, just earlier. That's **pull-forward**, not incremental revenue. Advanced attribution uses experiments or ML causal inference.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Measuring "revenue during promo" without comparing to baseline. Every promo looks successful because they run during peaks.

Correct Way: Always compare to a comparable baseline period. Uplift ratio $\neq$ raw revenue.

EXAM TIP CARD

Interview classic: *"How do you measure the effectiveness of a promotion?"* Winning answer: *"Compare revenue during the promo to a comparable baseline (7 days before, or same period year-ago). Uplift ratio = during / baseline.*

Also account for discount cost - a 1.2 $\times$ uplift may not offset a 30% discount."

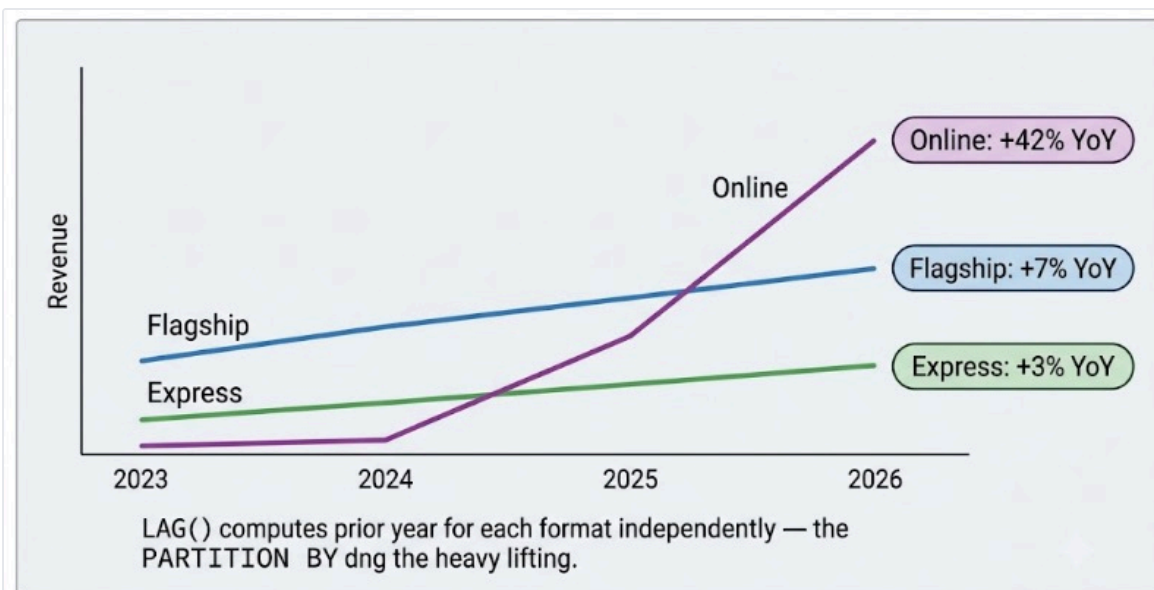
EXERCISE

Modify KPI 4 to also compute *net uplift dollars*: $(\text{during revenue} - \text{baseline revenue}) - \text{discount_amount}$. That's the actual profit contribution of the promo.

[AI Prompt] Paste this into any AI chatbot: *"For a retail promo effectiveness analysis, list 5 confounders that can make uplift look artificially good or bad. For each, explain how to control for it in SQL or analysis design."*

KPI 5 — YEAR-OVER-YEAR GROWTH BY STORE FORMAT

ChatGPT • AI Assistant



KPI 5 Result Shape — YoY Growth Ribbon Across Store Formats

KPI 5 Result Shape — YoY Growth Ribbon Across Store Formats


```
sql – KPI 5 – Year-over-year growth by store format
```

```
-- =====
-- KPI 5: YoY revenue growth by store format.
--      LAG() fetches prior-year revenue per format.
-- =====

WITH yearly_revenue AS (
    SELECT
        st.store_format,
        d.year,
        SUM(s.net_amount)          AS revenue,
        COUNT(DISTINCT s.order_id) AS orders,
        COUNT(DISTINCT s.customer_key) AS unique_customers
    FROM fact_sales s
    JOIN dim_store st ON st.store_key = s.store_key
    JOIN dim_date d ON d.date_key = s.date_key
    GROUP BY st.store_format, d.year
)
SELECT
    store_format,
    year,
    revenue,
    LAG(revenue) OVER (
        PARTITION BY store_format ORDER BY year
    ) AS
prev_year_revenue,
    ROUND(
        100.0 * (revenue - LAG(revenue) OVER (
            PARTITION BY store_format ORDER BY year
        )) / NULLIF(LAG(revenue) OVER (
            PARTITION BY store_format ORDER BY year
        ), 0),
        2
    ) AS
yoy_growth_pct,
    orders,
    unique_customers
FROM yearly_revenue
ORDER BY store_format, year;
```

How do I compute **year-over-year (YoY) growth** cleanly?

ChatGPT • AI Assistant

YoY comparisons are the classic use case for the `LAG()` window function. For each *(storeformat, year)* row, `LAG()` pulls the previous year's revenue from the row above (in year order, partitioned by storeformat). The percentage growth is then a simple arithmetic expression.

CONCEPT CARD

KPI 5 anatomy:

1. **Aggregate** - revenue by store_format × year.
2. **LAG()** - `LAG(revenue) OVER (PARTITION BY store_format ORDER BY year)` returns the prior year's revenue.
3. **YoY %** - `100 * (this_year - last_year) / last_year`.
4. **NULLIF guard** - prevents divide-by-zero for the first year.

Student

Why not just self-join on year - 1?

ChatGPT • AI Assistant

EXAMPLE CARD

`LAG()` beats self-join in three ways:

1. **One pass** - the engine reads each row once; self-join reads twice.
2. **Cleaner SQL** - one window function vs a WHERE clause with year arithmetic.
3. **Handles multi-year gaps** - `LAG()` with `LAG(revenue, 2)` gives you two years back; you can't do that easily in a self-join.
4. **Works with any partitioning** - swap `store_format` for `region` or `category` in one edit.

🗣️ ANALOGY

`LAG()` is like **peeking at the previous line of a book while reading**. Self-join is like **holding two copies of the book open at once**. Both work; peeking is easier.

⚠️ WARNING

`LAG()` returns `NULL` for the first row in each partition - that's why we use `NULLIF` in the denominator. Divide by `NULL` returns `NULL` - which SQL treats as "no growth to report," which is the correct semantic.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Hardcoding `WHERE year IN (2025, 2026)` and computing growth by hand. Doesn't scale to more years.

Correct Way: Use `LAG()` with `PARTITION BY` - one query works for any number of years.

EXAM TIP CARD

Interview favorite: *"Write a SQL query showing YoY revenue growth by store format."*
Winning answer: *"Use `LAG(revenue) OVER (PARTITION BY store_format ORDER BY year)` to fetch prior-year revenue, then compute the growth percentage."*

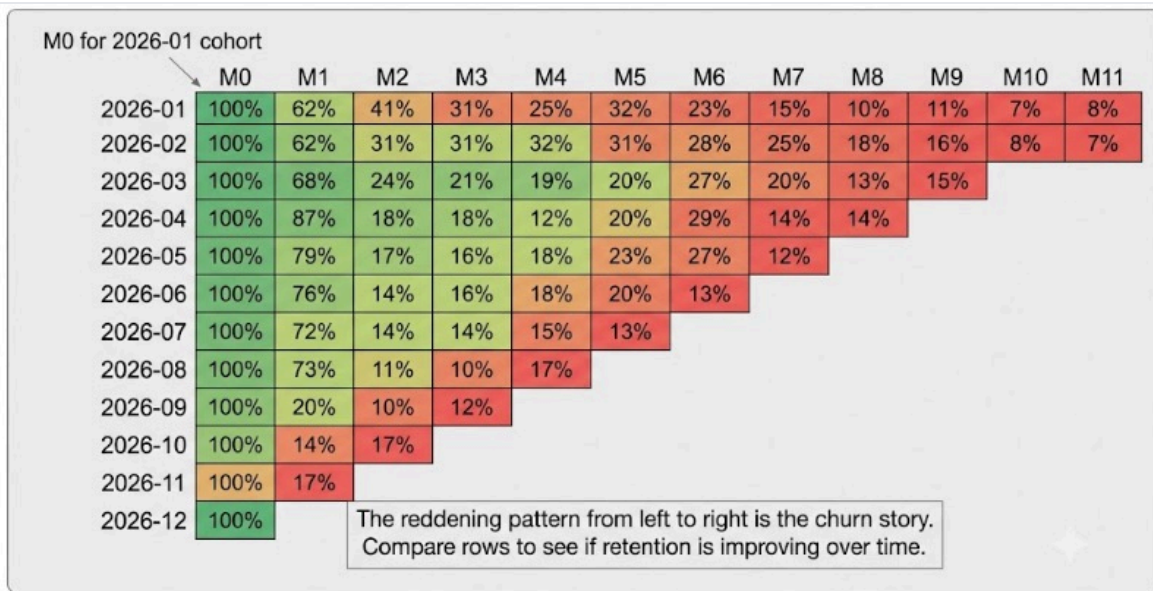
🔥 EXERCISE

Extend KPI 5 with quarter-over-quarter (QoQ) growth too. Change `PARTITION BY` to `store_format, year` and `ORDER BY quarter_number` - now you have both dimensions.

[AI Prompt] Paste this into any AI chatbot: *"Extend KPI 5 to compute both YoY and QoQ growth in the same query, plus a 3-year CAGR (compound annual growth rate) per store format. Explain each window function used."*

KPI 6 — COHORT RETENTION

ChatGPT • AI Assistant



KPI 6 Result Shape — The Classic Cohort Retention Heatmap.

KPI 6 Result Shape — The Classic Cohort Retention Heatmap

sql – KPI 6 – Monthly cohort retention (the classic ML/growth query)

```
-- =====
-- KPI 6: Monthly cohort retention.
--       For each cohort (month of first order), what % of
--       the cohort's customers are still active N months later?
-- =====

WITH first_orders AS (
  -- Layer 1: define each customer's cohort by their first order
  SELECT
    s.customer_key,
    MIN(d.full_date)                                AS first_order_date,
    DATE_TRUNC('month', MIN(d.full_date))           AS cohort_month
  FROM fact_sales s
  JOIN dim_date   d ON d.date_key = s.date_key
  GROUP BY s.customer_key
),
orders_with_cohort AS (
  -- Layer 2: tag every order with (cohort, months_since_cohort)
  SELECT
    fo.cohort_month,
    s.customer_key,
    DATE_TRUNC('month', d.full_date)                AS order_month,
    (DATE_PART('year', d.full_date) * 12 + DATE_PART('month',
d.full_date))
    - (DATE_PART('year', fo.cohort_month) * 12 + DATE_PART('month',
fo.cohort_month))
```

```

AS months_since_cohort

FROM fact_sales s
JOIN first_orders fo ON fo.customer_key = s.customer_key
JOIN dim_date      d ON d.date_key      = s.date_key
),
cohort_sizes AS (
  -- Layer 3: cohort denominator
  SELECT cohort_month, COUNT(DISTINCT customer_key) AS cohort_size
  FROM    first_orders
  GROUP BY cohort_month
)
-- Final: retention percentage per (cohort, month_offset)
SELECT
  o.cohort_month,
  c.cohort_size,
  o.months_since_cohort,
  COUNT(DISTINCT o.customer_key) AS
active_customers,
  ROUND(100.0 * COUNT(DISTINCT o.customer_key) / c.cohort_size, 1) AS
retention_pct
FROM  orders_with_cohort o
JOIN  cohort_sizes      c ON c.cohort_month = o.cohort_month
WHERE o.cohort_month ≥ DATE '2026-01-01'
      AND o.months_since_cohort ≤ 11
GROUP BY o.cohort_month, c.cohort_size, o.months_since_cohort
ORDER BY o.cohort_month, o.months_since_cohort;

```

Student

How do I compute **cohort retention** - the classic "how many customers who signed up in month X are still buying N months later?"

ChatGPT • AI Assistant

Cohort retention is the most involved query in this lab. Three layers of CTE:

1. **first_orders** - each customer's first order date and their cohort month.
2. **orders_with_cohort** - every order tagged with the customer's cohort AND the months since their cohort start.
3. **cohort_sizes** - how many customers total joined each cohort.

Final query joins them to compute retention percentage per (cohort, month_offset) cell - the retention heatmap.

CONCEPT CARD

Cohort retention query pattern:

1. **Define cohort** - for each customer, the month of their FIRST order (`MIN(order_date)`).
2. **Tag every subsequent order** with (cohortmonth, monthssince_cohort).
3. **Count active customers per cohort per month_offset.**
4. **Divide by cohort size** to get retention %.
5. **Pivot** in BI tool for the classic triangular heatmap.

Student

Why is this SO much harder than the other KPIs?

ChatGPT • AI Assistant

EXAMPLE CARD

Cohort retention is hard because:

1. It requires a **self-derived cohort** - the cohort isn't in the schema; you compute it.
2. It uses **relative time** - "months since cohort start," not absolute months.
3. It requires **two joins** - customer to their cohort, then order to (cohort, offset).
4. It creates a **triangular result** - earlier cohorts have more months of history than newer ones.
5. This one query is 90% of the churn/growth analytics you'll ever write.

🔗 ANALOGY

Cohort retention is like **tracking a graduating class through their careers**. The class of 2020 has 5 years of career data; the class of 2024 has 1. Both are on the same chart, staggered by graduation year.

⚠️ WARNING

For a business with 50K new customers per month across 3 years, the query joins $50K \times 12 \times 3 = 1.8M$ rows per cohort. Filter early (WHERE cohortmonth >= '2024-01-01') and materialize into an `aggcohort\_retention` table for dashboard consumption.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Using SIGNUP date as the cohort anchor. Some customers sign up but never buy - they distort retention. Use FIRST ORDER date instead.

Correct Way: Cohort = month of first order. Retention = active in month N counted against the cohort size.

EXAM TIP CARD

Interview classic: *"Write a SQL query showing monthly cohort retention for the last 12 months."* Winning approach: *"Three CTEs - first_orders (defines cohorts), orders_with_cohort (tags orders with offset), cohort_sizes (denominator). Final query is a JOIN + GROUP BY."*

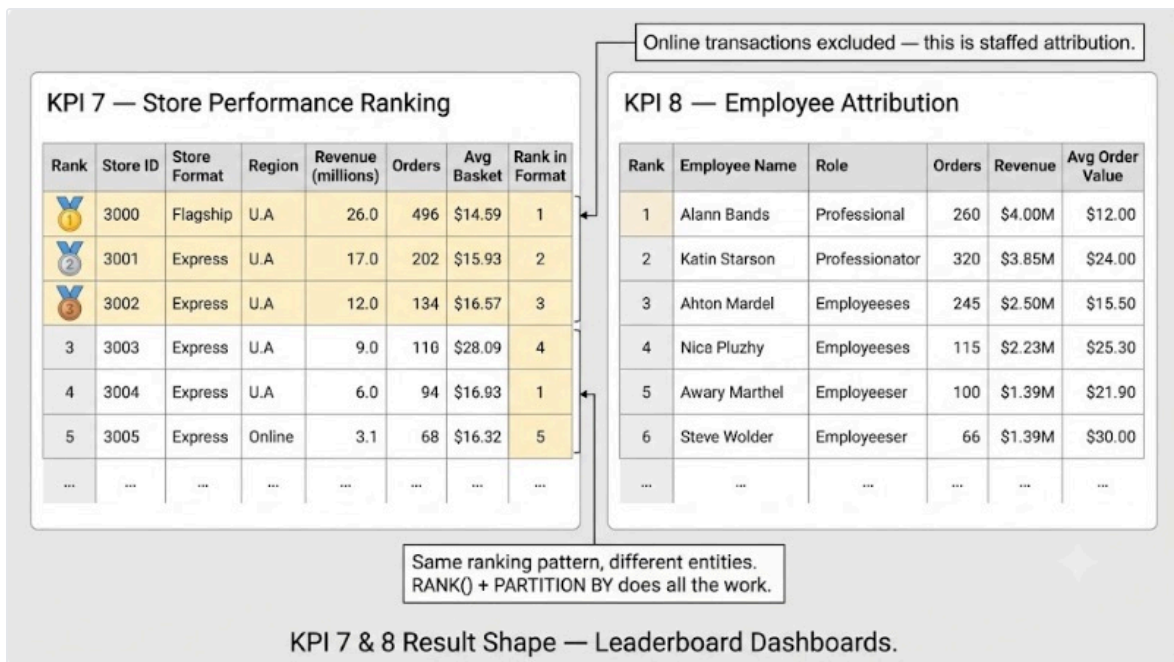
🔥 EXERCISE

Modify KPI 6 to compute WEEKLY cohorts and WEEKLY retention offsets - useful for a fast-moving subscription business. Just swap `DATE_TRUNC('month', ...)` for `DATE_TRUNC('week', ...)`.

[AI Prompt] Paste this into any AI chatbot: *"Extend the cohort retention query to also compute cumulative revenue per cohort (not just retention). Show how AVG revenue per active customer trends by cohort age."*

KPI 7 & 8 — STORE PERFORMANCE + EMPLOYEE ATTRIBUTION

ChatGPT • AI Assistant



KPI 7 & 8 Result Shape — Leaderboard Dashboards

sql – KPI 7 – Store performance ranking (overall + within format)

```
-- =====
-- KPI 7: Store performance ranking with sub-ranking by format.
-- =====

WITH store_totals AS (
    SELECT
        st.store_id, st.store_name, st.store_format, st.region,
        SUM(s.net_amount) AS revenue,
        COUNT(DISTINCT s.order_id) AS orders,
        COUNT(DISTINCT s.customer_key) AS
unique_customers,
        SUM(s.net_amount) / NULLIF(COUNT(DISTINCT s.order_id), 0) AS
avg_basket_value
    FROM fact_sales s
    JOIN dim_store st ON st.store_key = s.store_key
    JOIN dim_date d ON d.date_key = s.date_key
    WHERE d.year = 2026
        AND st.is_active = TRUE
    GROUP BY st.store_id, st.store_name, st.store_format, st.region
)
SELECT
    store_id, store_name, store_format, region,
    revenue, orders, unique_customers,
    ROUND(avg_basket_value, 2) AS
avg_basket,
    RANK() OVER (ORDER BY revenue DESC) AS
```



```

overall_rank,
    RANK() OVER (PARTITION BY store_format ORDER BY revenue DESC) AS
rank_in_format,
    RANK() OVER (PARTITION BY region          ORDER BY revenue DESC) AS
rank_in_region
FROM    store_totals
ORDER  BY overall_rank
LIMIT  20;

```

sql – KPI 8 – Employee sales attribution (staffed sales only)

```

-- =====
-- KPI 8: Employee sales attribution.
--      Online (employee_key IS NULL) excluded – staffed only.
-- =====
SELECT
    e.employee_id,
    e.full_name,
    e.role,
    e.department,
    COUNT(DISTINCT s.order_id)                AS orders,
    SUM(s.net_amount)                        AS revenue,
    ROUND(SUM(s.net_amount) / NULLIF(COUNT(DISTINCT s.order_id), 0), 2)
AS avg_order_value,
    RANK() OVER (ORDER BY SUM(s.net_amount) DESC)      AS
revenue_rank,
    RANK() OVER (
        PARTITION BY e.department ORDER BY SUM(s.net_amount) DESC
    )                                                  AS
rank_in_department
FROM fact_sales    s
JOIN dim_employee  e ON e.employee_key = s.employee_key
JOIN dim_date      d ON d.date_key      = s.date_key
WHERE d.year        = 2026
      AND s.employee_key IS NOT NULL
      AND e.is_current = TRUE
GROUP BY e.employee_id, e.full_name, e.role, e.department
HAVING COUNT(DISTINCT s.order_id) ≥ 50  -- meaningful sample size
ORDER BY revenue DESC
LIMIT  25;

```

Show me the **store performance ranking** and the **employee attribution** queries.

ChatGPT • AI Assistant

Both KPIs are variations of the same pattern: aggregate by entity, rank by revenue, add secondary metrics. KPI 7 ranks stores; KPI 8 ranks employees. Both use `RANK()` `OVER (ORDER BY revenue DESC)` - with an optional `PARTITION BY` for within-group rankings.

CONCEPT CARD

Shared pattern for both KPIs:

1. **Aggregate** - revenue, orders, unique customers, avg basket value per entity.
2. **Rank** - overall AND within-group (per store format, per employee role).
3. **Filter** - top N by rank.
4. **Filter Type 2 dims** - `is_current = TRUE` for latest attributes.

Student

How do I handle the fact that online orders have no employee?

ChatGPT • AI Assistant

EXAMPLE CARD

The `employee_key` in `fact_sales` is nullable - because online transactions have no cashier. Two handling strategies:

1. **Exclude nulls** - `WHERE s.employee_key IS NOT NULL` (KPI 8 focuses only on staffed sales).
2. **Include as "online"** - `COALESCE(e.full_name, 'Online Sale')` to bucket them.

KPI 8 excludes nulls because leadership only wants staffed-attendant attribution. Marketing might use the other approach.

🔗 ANALOGY

Ranking stores and employees is like **the leaderboard at a fitness challenge**. You show total distance, weekly average, and rank - with the option to filter by group (weight class, department). Same pattern, different entities.

⚠️ WARNING

Small stores or new employees can look great on `avg_basket_value` because they have few transactions - one big order distorts the average. Add a `HAVING COUNT(*) ≥ N` filter for meaningful rankings.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Ranking employees without joining `is_current = TRUE` - you get one row per employee version, splitting their revenue across historical versions.

Correct Way: GROUP BY the natural key `employee_id` ; filter `is_current = TRUE` on the dim JOIN.

EXAM TIP CARD

Interview classic: "Write a SQL query showing the top 5 employees in each department by revenue." Winning approach: "Use `DENSE_RANK() OVER (PARTITION BY department ORDER BY revenue DESC)` in a CTE and filter `WHERE rank ≤ 5`."

🔥 EXERCISE

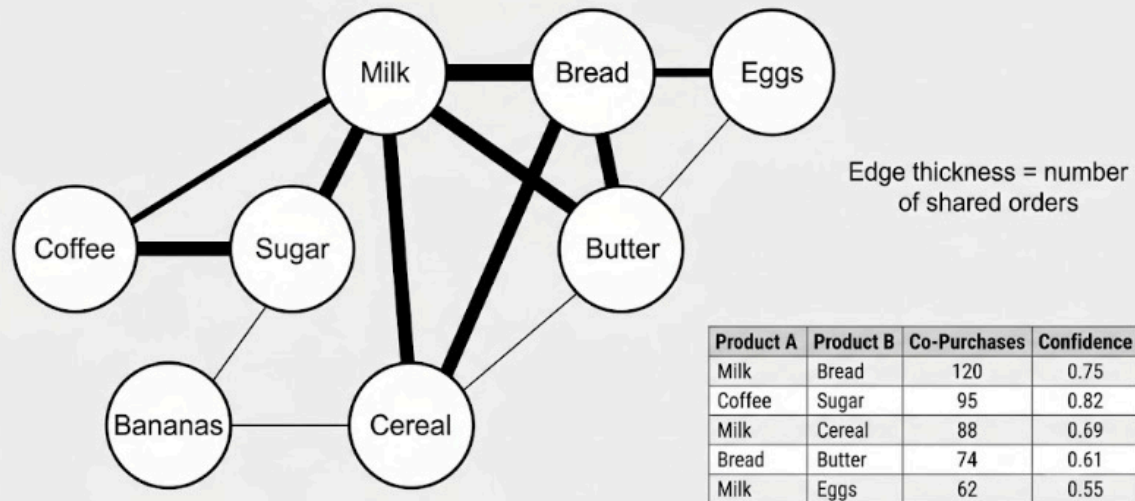
Modify KPI 7 to also rank stores WITHIN their region. Modify KPI 8 to show employees who have both high revenue AND high customer count (well-rounded performers).

[AI Prompt] Paste this into any AI chatbot: "Extend KPI 8 (employee attribution) to compute a composite performance score: 40% revenue rank + 30% customer count rank + 30% avg basket value rank. Show how to combine multiple RANK() outputs into one score."

KPI 9 — MARKET BASKET ANALYSIS



ChatGPT • AI Assistant

KPI 9 Result Shape — Co-Purchase Graph.

A self-join reveals affinity. The heavy edges point at bundling opportunities.

KPI 9 Result Shape — Co-Purchase Graph

sql – KPI 9 – Market basket analysis (co-purchase counts)

```
-- =====
-- KPI 9: Market basket analysis – top co-purchased product pairs.
-- =====

WITH order_products AS (
  -- One row per (order, product) – deduped
  SELECT DISTINCT
    s.order_id,
    p.sku,
    p.product_name,
    p.category
  FROM fact_sales s
  JOIN dim_product p ON p.product_key = s.product_key
  JOIN dim_date d ON d.date_key = s.date_key
  WHERE d.year = 2026
  AND p.is_current = TRUE
),
pairs AS (
  -- Self-join: pair every product with every OTHER product in the same order
  -- a.sku < b.sku ensures one direction only, no self-pairs, no
  duplicates
  SELECT
    a.sku AS product_a_sku,
    a.product_name AS product_a_name,
    a.category AS product_a_category,
```

```

    b.sku          AS product_b_sku,
    b.product_name AS product_b_name,
    b.category     AS product_b_category,
    COUNT(*)       AS co_purchases
FROM order_products a
JOIN order_products b
    ON a.order_id = b.order_id
   AND a.sku      < b.sku
GROUP BY a.sku, a.product_name, a.category,
         b.sku, b.product_name, b.category
HAVING COUNT(*) ≥ 100      -- filter noise
)
SELECT *
FROM   pairs
ORDER BY co_purchases DESC
LIMIT 20;

```

Student

Now for the fun one - **what products are bought together?**

ChatGPT • AI Assistant

Market basket analysis finds product pairs that co-occur in the same order. The SQL is a **self-join on `order\_id`** - each order becomes pairs of its line items, filtered to avoid duplicates and self-pairs. Aggregate the pair counts and you have a co-purchase leaderboard.

CONCEPT CARD

KPI 9 mechanics:

1. **CTE** - *distinct (orderid, product) pairs from `factsales`.*
2. **Self-join** - pair every product with every other product in the same order.
3. **Deduplication** - `a.sku < b.sku` to avoid (A,B) and (B,A) duplicates, and to exclude self-pairs.
4. **Aggregate** - count how often each pair appears together.
5. **Filter** - `HAVING COUNT(*) ≥ 100` to skip rare pairs.

Student

How do I go beyond raw counts to real market basket metrics?

ChatGPT • AI Assistant

EXAMPLE CARD

Two additional metrics used in real analysis:

1. **Support** - how often the pair appears in ALL orders = $\text{COUNT}(\text{pair}) / \text{total_orders}$.
2. **Confidence** - probability of buying B given A = $\text{COUNT}(\text{pair}) / \text{COUNT}(A)$.
3. **Lift** - how much MORE likely they're bought together than by chance = $\text{confidence} / P(B)$.
4. Lift > 1 means real association; Lift < 1 means the products actually repel each other.

Production analyses build the association-rule triple (support, confidence, lift) - often via specialized libraries or ML tooling.

🔗 ANALOGY

Market basket analysis is like **seeing which items appear on the same restaurant orders**. Discovering that 80% of orders with fries also have a drink tells the kitchen (and marketing) to bundle them.

⚠️ WARNING

Self-joins on huge fact tables are expensive. For a fact table with 10M line items across 2.5M orders, this query touches billions of row pairs. Restrict to a specific time window and top-N products to make it tractable.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: $a.sku \diamond b.sku$ - excludes self-pairs but still creates (A,B) AND (B,A) duplicates.

Correct Way: $a.sku < b.sku$ - one direction only; excludes self-pairs AND duplicates.

EXAM TIP CARD

Interview favorite: *"How would you find products frequently bought together?"*

Winning answer: *"Self-join the fact table on order_id with 'a.sku < b.sku', count occurrences of each pair, and rank. For serious analysis, compute support, confidence, and lift."*

EXERCISE

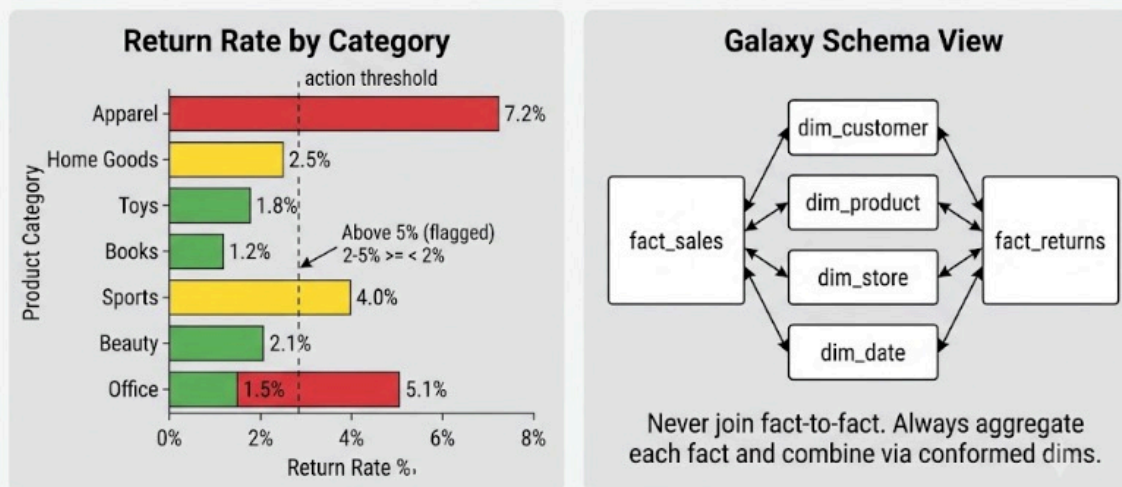
Extend KPI 9 to also compute lift: for each pair (A, B), divide P(both in same order) by $P(A) \times P(B)$. Lift > 1.5 = strong association.

[AI Prompt] Paste this into any AI chatbot: "Extend KPI 9 to compute support, confidence, and lift for the top 20 product pairs at Mango Mart. Explain each metric and what makes a 'strong' association rule."

KPI 10 — RETURN RATES + PERFORMANCE TIPS

ChatGPT • AI Assistant

KPI 10 Result Shape — Return Rates + Galaxy Schema View.



One category insight, one schema pattern that will host every future business process.

KPI 10 Result Shape — Return Rates + Galaxy Schema View

sql — KPI 10 — Return rates by category (galaxy schema pattern)

```

-- =====
-- KPI 10: Return rates by product category.
--       Requires fact_returns as a peer fact table with the
--       same conformed dims as fact_sales.
-- =====

-- ADD fact_returns TO THE WAREHOUSE
CREATE TABLE fact_returns (
    returns_key      BIGINT PRIMARY KEY,
    date_key         INT    NOT NULL REFERENCES dim_date(date_key),

```

```

    customer_key      BIGINT NOT NULL REFERENCES
dim_customer(customer_key),
    product_key       BIGINT NOT NULL REFERENCES
dim_product(product_key),
    store_key         BIGINT NOT NULL REFERENCES dim_store(store_key),
    original_order_id VARCHAR(50) NOT NULL,
    return_id         VARCHAR(50) NOT NULL,
    quantity_returned INT      NOT NULL,
    refund_amount     DECIMAL(12, 2) NOT NULL,
    restocking_fee    DECIMAL(12, 2) DEFAULT 0,
    return_reason     VARCHAR(50)
);

CREATE INDEX idx_fr_date      ON fact_returns(date_key);
CREATE INDEX idx_fr_customer ON fact_returns(customer_key);
CREATE INDEX idx_fr_product  ON fact_returns(product_key);

-- =====
-- THE KPI QUERY: return rate by category
-- Notice: fact_sales and fact_returns are aggregated separately,
-- then joined via a shared category attribute (dim_product).
-- =====

WITH sales_by_cat AS (
    SELECT
        p.category,
        SUM(s.net_amount) AS sales_amount,
        SUM(s.quantity)   AS units_sold
    FROM fact_sales s
    JOIN dim_product p ON p.product_key = s.product_key
    JOIN dim_date     d ON d.date_key   = s.date_key
    WHERE d.year      = 2026
        AND p.is_current = TRUE
    GROUP BY p.category
),
returns_by_cat AS (
    SELECT
        p.category,
        SUM(r.refund_amount) AS refund_amount,
        SUM(r.quantity_returned) AS units_returned
    FROM fact_returns r
    JOIN dim_product p ON p.product_key = r.product_key
    JOIN dim_date     d ON d.date_key   = r.date_key
    WHERE d.year      = 2026
        AND p.is_current = TRUE
    GROUP BY p.category
)

```



```

SELECT
    s.category,
    s.sales_amount,
    s.units_sold,
    COALESCE(r.refund_amount, 0) AS refund_amount,
    COALESCE(r.units_returned, 0) AS units_returned,
    ROUND(
        100.0 * COALESCE(r.refund_amount, 0) / NULLIF(s.sales_amount, 0),
        2
    ) AS refund_rate_pct,
    ROUND(
        100.0 * COALESCE(r.units_returned, 0) / NULLIF(s.units_sold, 0),
        2
    ) AS unit_return_rate_pct
FROM    sales_by_cat s
LEFT JOIN returns_by_cat r ON r.category = s.category
ORDER BY refund_rate_pct DESC;

```

Student

The last KPI - **return rates by category** - plus tips to make all these queries production-ready.

ChatGPT • AI Assistant

KPI 10 introduces **a new fact table** to Mango Mart's warehouse: **fact_returns**. Returns are a separate business process (Chapter 2's rule) with their own grain - one row per returned line item.

They share **dim_customer**, **dim_product**, **dim_store**, and **dim_date** with **fact_sales** (conformed dims from Chapter 4's galaxy pattern).

CONCEPT CARD

KPI 10 architecture:

1. **Add 'fact_returns'** - grain: one row per returned line item; measures: *refundamount*, *quantityreturned*.
2. **Two-CTE pattern** - *salesbycategory* + *returnsbycategory*.
3. **LEFT JOIN** - categories with sales but no returns still show up (return rate = 0).
4. **Return rate** = $\text{refundamount} \div \text{salesamount}$, expressed as %.

What are the top **performance tips** for all these KPI queries?

ChatGPT • AI Assistant

EXAMPLE CARD

Six production-ready patterns applied throughout this lab:

1. **Filter through dim_date** - enables partition pruning.
2. **Filter `is\_current = TRUE` early** - trims Type 2 dims to 1 row per natural key.
3. **Aggregate before joining** where possible - reduces intermediate volume.
4. **Materialize** the top-3 KPIs into `agg_ ...` tables refreshed nightly - dashboards hit tiny aggregates, not billions of fact rows.
5. **Column selection** - never `SELECT *` in column stores (Chapter 5's warning applies here).
6. **Explain plans** - periodically run `EXPLAIN` on production queries; watch for full-scans that should be partition-pruned.

🔗 ANALOGY

Return rate analysis is like **the customer complaint hotline for a product line**. High rates are the loudest signal that something is wrong - quality, description, sizing. The dashboard should surface top offenders every morning.

⚠️ WARNING

Return rates only tell part of the story. A high rate on a low-volume product isn't necessarily bad - the sample size may be too small. Always show volume alongside rate.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Joining `fact_sales` directly to `fact_returns` on `order_id` and `sku`. Fact-to-fact joins are the anti-pattern Chapter 4 warned about.

Correct Way: Aggregate each fact separately, then join the aggregates via conformed dimension attributes.

EXAM TIP CARD

Interview classic: *"How would you compute return rates by category if sales and returns are separate fact tables?"* Winning answer: *"Aggregate each fact independently by category (via dim_product), then join the two aggregates. Never join fact-to-fact directly - always go through conformed dimensions."*

 EXERCISE

Add `fact_returns` DDL to your Mango Mart schema. Populate 10-20 sample return rows. Run KPI 10 - verify return-rate calculations against your inserts.

[AI Prompt] Paste this into any AI chatbot: *"Design factreturns DDL for a retail warehouse. Include grain, FKs to conformed dims (customer, product, store, date), and measures (refundamount, quantityreturned, restockingfee)."*

Show a query joining *factsales* and *factreturns* to compute return rates by product."

LAB REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

You just wrote 10 real KPI queries against the Mango Mart star schema, covering every core technique of analytical SQL. Every question the executive team asked in Lab 1's business scenario now has a working query behind it.

- LAB SETUP: Recap of the six-dim Mango Mart schema plus five query conventions - *dimdate-driven time filtering*, *iscurrent* on Type 2 dims, *net_amount* for revenue, consistent aliases, explicit column lists.
- KPI 1 & 2 - REVENUE ANALYTICS: Three-way join + GROUP BY for revenue by category × region × month; RANK() window for top products by revenue AND units in one query.
- KPI 3 - CUSTOMER LIFETIME VALUE: Two-layer CTE - per-customer LTV then per-segment aggregation with PERCENTILE_CONT for the honest median.
- KPI 4 - PROMO EFFECTIVENESS: Filtered aggregates for before/during/after revenue, uplift ratio, and discount cost - with a caution about pull-forward and confounders.
- KPI 5 - YEAR-OVER-YEAR GROWTH: LAG() window function with PARTITION BY store_format ORDER BY year - clean multi-year growth in a single pass.

- KPI 6 - COHORT RETENTION: Three-CTE pattern (*firstorders*, *orderswithcohort*, *cohortsizes*) producing the classic triangular retention heatmap.
- KPI 7 & 8 - STORE + EMPLOYEE RANKINGS: `RANK()` OVER (ORDER BY revenue DESC) for overall ranks plus PARTITION BY for within-group rankings. Null handling for online (unstaffed) sales.
- KPI 9 - MARKET BASKET ANALYSIS: Self-join on `order_id` with `a.sku < b.sku` to find product pairs; support/confidence/lift as production extensions.
- KPI 10 + PERFORMANCE TIPS: Introduced *factreturns as a peer fact table*, *aggregated each fact separately*, *joined via conformed dims*. Six production patterns: *dimdate* filtering, *is_current* early, aggregate before join, materialized aggregates, explicit columns, cross-fact via conformed dims.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You've now walked the full path - from raw data-modeling foundations in Chapter 1 to a working analytics warehouse with 10 live KPIs in Lab 2. Every query in this lab is production-shaped: it uses the conformed dim pattern, respects SCD Type 2, filters partitionable columns, and would run in Snowflake, BigQuery, or any modern warehouse without modification.

A senior data engineer's first year of KPI work is largely variations on these ten queries. The variations aren't harder than what you've seen - they're recombinations of the patterns here.

Once your muscle memory locks in the shape *"SELECT dim-attributes, aggregated-measures FROM fact JOIN dim WHERE dim-filters GROUP BY dim-attributes ORDER BY measure"*, you can answer any business question in minutes.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

Congratulations - you've completed the Refcard. Go build something.